

MEMO-Appflow: 从 App 与操作系统证据到真实应用推荐和系统调度

Jingyi Guo

2026-07-02

摘要

MEMO-Appflow 是面向 Android 的端侧应用预测与系统调度系统。它不只观察用户打开了哪些应用,也在手机内部持续采集 eBPF 内核证据和系统状态,将 app 序列、网络、Binder、内存、显示、媒体和进程信号按时间窗口压缩为 MAPLE 场景输入。端侧 MAPLE 模型据此预测接下来可能使用的 Top-3 真实应用,并输出 `scheduler_bits` 选择预先实现的系统调度动作。Android 端随后更新桌面 Widget,并执行 warm launch、memory trim、service manager refresh 等动作。本文按照最终成功的系统版本组织,说明端侧流水线、数据结构、模型接口、动作执行和实验结果。真实 Pixel 5 实验显示,Top-3 推荐可以随 3 分钟窗口更新;30 条真实记录预测实验中,app+OS/eBPF 配置在 Top-3 Hit、F1@3 和 MRR 上高于 app-only baseline;系统调度性能实验中,Stage 3 调度使同一 Top-1 应用 QQ 的 Launch TotalTime 从 433 ms 降至 168 ms, WaitTime 从 440 ms 降至 173 ms,综合压力分数从 422.56 降至 9.71。

目录

1 引言与问题定义	3
2 端侧产品架构	3
2.1 闭环链路	3
2.2 实时窗口与任务队列	4
3 数据结构与压缩	4
3.1 采集对象	4
3.2 时间对齐与重复压缩	6

4	MAPLE 推理与系统动作	6
4.1	MAPLE 输入输出	6
4.2	真实应用映射与 Widget	7
4.3	ActionExecutor 与 scheduler bits	7
5	实验设计	7
6	结果与分析	8
6.1	实时闭环结果	8
6.2	预测消融结果	9
6.3	系统调度性能结果	9
7	实现对应关系	9
8	局限与下一步	11
9	结论	11

1 引言与问题定义

移动设备的性能体验来自两条线索：一条是用户接下来会打开什么 app，另一条是操作系统此刻是否有足够资源承接这个行为。传统应用预测通常只把问题建模成 app 序列预测，例如根据最近打开过的应用推荐下一个应用。这种方法可以得到一个 Top-3 列表，但它看不到更深层的系统状态：网络是否活跃、Binder 服务是否频繁、内存是否正在回收、显示线程是否繁忙、媒体解码是否正在发生。对于一个真正需要优化手机体验的系统，仅有 app 序列是不够的。

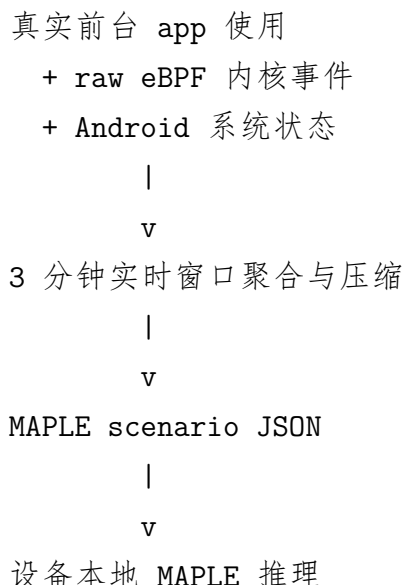
MEMO-Appflow 的核心思路是把 app 侧和 OS 侧信息放在同一条端侧闭环中。输入端同时收集真实前台 app 序列、eBPF 内核事件和 Android 系统状态；推理端用本地 MAPLE 模型预测用户接下来可能使用的 Top-3 真实应用，并判断系统应该执行哪些调度动作；输出端既更新桌面 Widget，让推荐成为可点击入口，也通过 ActionExecutor 执行系统动作，影响内存、预热、服务刷新和调度策略。概括来说，系统完成的是“从 app + OS 到 app + OS”：用 app 行为和 OS 证据共同赋能 app 推荐和 OS 性能优化。

这套系统的工程目标不是在电脑上跑一组脚本，而是让功能尽可能在 Android 设备内部闭环运行。Host 可以负责安装 APK、推送模型文件和拉取日志，但不能承担采集、解析、推理或动作执行的产品逻辑。这样设计的原因很直接：如果系统需要一直连着电脑才能工作，它就不是可以迁移到真实手机上的产品。

2 端侧产品架构

2.1 闭环链路

当前 MEMO-Appflow 的端侧链路如下：



```

      |
      v
Top-3 app id/category + scheduler_bits
      |
      v
真实 Android app 映射 + ActionExecutor
      |
      v
桌面 Widget 更新 + 系统调度动作

```

这条链路中，用户看到的是 Top-3 真实应用和系统优化状态；eBPF、Binder、Surface-Flinger、进程调度等信息不会直接堆在普通界面上，而是保存在报告和 debug 入口中。这样既保留了实验可追溯性，也避免把系统内部概念暴露成用户需要理解的产品功能。

2.2 实时窗口与任务队列

实时模式使用固定 3 分钟窗口。一个窗口结束后，系统将该窗口里的 app 序列、eBPF 事件和系统状态压缩成 scenario，并提交给唯一的 MAPLE worker；与此同时，下一个窗口继续采集。推理和采集解耦后，MAPLE 的 20-40 秒端侧推理耗时不会阻塞用户继续使用手机，也不会造成“收完一段、停下来推理、再继续收”的断点。

为了避免长期后台运行时线程和任务堆积，系统只保留一个 collector loop、一个 MAPLE worker 和一个 latest pending slot。如果 MAPLE 推理慢于窗口节奏，系统不会无限排队旧窗口，而是保留最新待推理窗口。当前窗口仍然是主输入；EMA memory 只保存有界历史偏好，作为辅助上下文，而不是无限增长的日志。

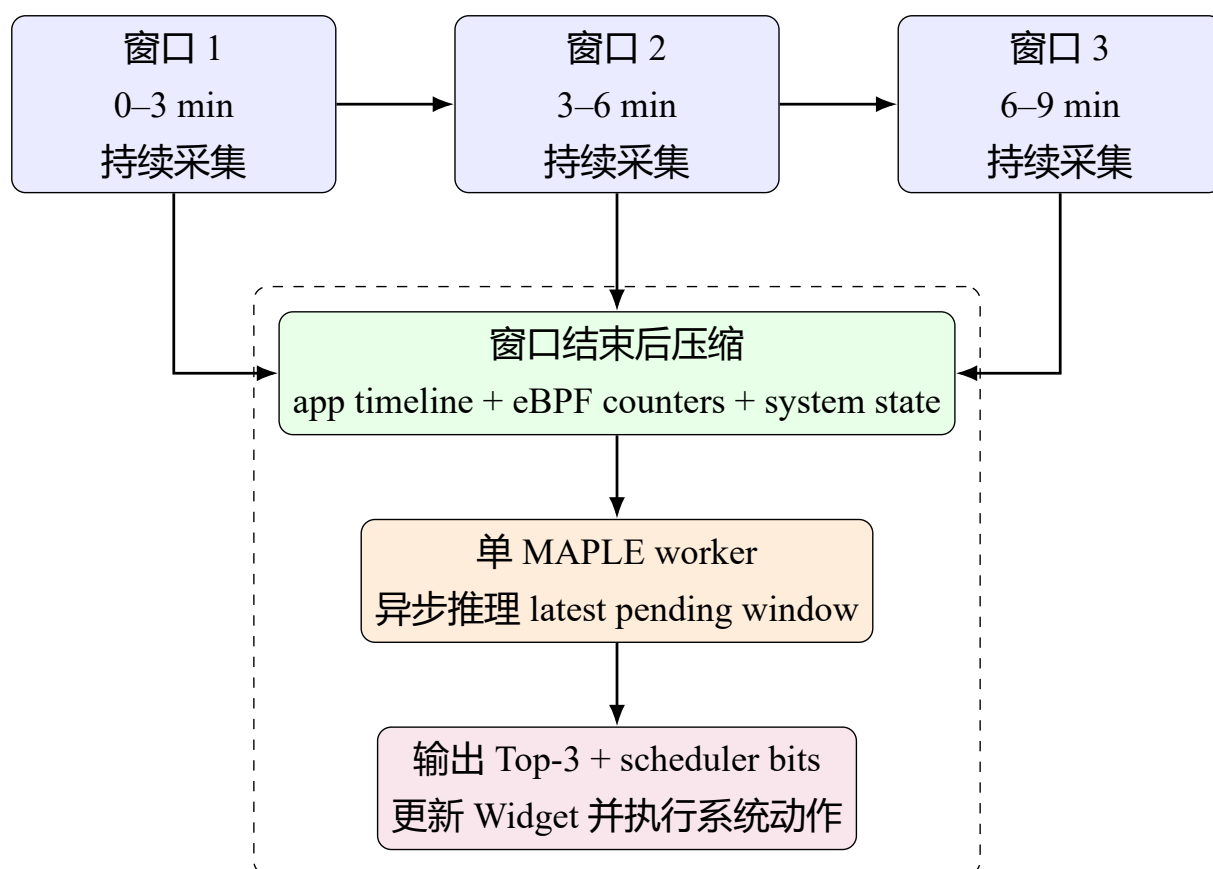
3 数据结构与压缩

3.1 采集对象

MAPLE 的输入不是单一 eBPF 日志，而是一个对齐后的 context JSON。它主要包含三类信息。

App 侧信息记录前台 app 序列，包括包名、应用名、切换时间和持续时间。它回答“刚才真实用了哪些 app”。这部分对于 Top-3 应用预测是必要的，因为用户行为本身有连续性。

OS 侧信息来自 eBPF 和系统状态采样。eBPF 事件包括网络 sendto/recvfrom、文件 openat、Binder/服务调用、进程生命周期、调度和运行时信号。系统状态包括 memory、



推理在后台执行；下一窗口继续采集；只保留单 worker 和最新待推理窗口

图 1: 3 分钟采集窗口与 MAPLE 异步推理流水线

battery、network、camera/media、display/UI、process/service 等。它回答“这些 app 使用发生时，系统内部处在什么状态”。

可执行动作契约列出 Android 端已经实现的调度动作。MAPLE 不直接输出 shell 命令，而是在这组动作中用 0/1 bitmask 选择是否执行，降低模型直接控制系统的风险。

3.2 时间对齐与重复压缩

仅有 app 序列或仅有 eBPF 都不够。一个合理输入应该告诉模型：同一个时间段里，哪个 app 在前台，OS 内部发生了哪些事件，事件强度是多少。因此系统按时间窗口把 app segment 和 eBPF counter 对齐。比如某个窗口中 Chrome 在前台，同时出现大量 recvfrom 和 display refresh，模型就能理解这是浏览/媒体/网络相关场景，而不是孤立地看到一串网络系统调用。

原始 eBPF trace 会有大量重复行。如果把重复事件逐条送进 prompt，模型上下文会被低信息密度文本占满。MEMO 的压缩方式是把同一个窗口内相同类型、相同证据类别、相近进程来源的事件合并成 counter key，并保留 count、时间范围和代表性样本。例如上千条 MEMO_RECVFROM 会压缩成“Network IO 在该窗口重复 N 次，主要关联哪些进程和前台 app”。这样会损失单条事件细节，但保留调度最需要的信息：发生了什么、发生在哪个窗口、强度有多高、和哪个 app 使用阶段相关。

4 MAPLE 推理与系统动作

4.1 MAPLE 输入输出

MAPLE 是唯一预测入口。Android 端构造 scenario JSON 后，本地调用 MAPLE/llama.cpp 可执行文件和 GGUF 模型。Prompt 明确要求预测“用户接下来可能使用的三个 app”，同时输出系统调度 bitmask。最终输出分为三层：

- Stage 1 预测用户可见 app 类别，例如 Communication、Media、Browser、Camera。
- Stage 2 在类别候选中选择 MAPLE app id，再由 Android 映射成真实包名。
- Stage 3 输出 scheduler_bits，选择系统动作。

这里需要区分 app prediction candidates 和 scheduler evidence。Memory Management、Binder、process runtime、system service 这类 OS-only 类别可以影响调度，但不能被当成“下一个 app 类别”。否则小模型容易把系统资源类别误预测为用户要点击的应用类别。当前版本将用户可见 app 类别和 OS 调度证据分开：前者用于 Top-3，后者用于 scheduler_bits 和安全策略。

4.2 真实应用映射与 Widget

MAPLE 输出的 App id 或类别不能直接展示给用户。AppIdMapping 使用 PackageManager 扫描设备上的 launchable apps，读取真实包名、label、launch intent 和可用类别，再根据 MAPLE 类别、历史使用、包名/label 关键词和设备实际安装情况排序。这样迁移到另一台手机时不需要硬编码“必须有某个 app”：如果通信类场景中没有 QQ，但有微信、短信或电话，Communication 类候选会自然落到那台设备上真实存在且可启动的应用。

桌面 Widget 是产品出口。实时模式每 3 分钟刷新 Top-3 后，Widget 展示真实 app 图标、名称和更新时间，点击图标可以直接打开应用。App 内部则展示更详细的系统判断、证据摘要、动作日志和报告入口。

4.3 ActionExecutor 与 scheduler bits

ActionExecutor 把 MAPLE 的 bitmask 转成系统动作。当前定义 9 个动作位。表中使用动作简称；代码中对应的完整 action id 保存在‘ActionExecutor.kt’的动作表中。

表 1: scheduler bits 对应的 Android 端动作

bit	动作简称	含义
0	warm_top1	预热 Top-1 推荐应用并回到 HOME。
1	warm_top2_idle	设备空闲时预热 Top-2。
2	memory_trim	对低优先级候选应用做 memory trim。
3	kill_background	关闭被选中的低优先级后台包。
4	drop_cache	仅在 critical memory 压力下 drop cache。
5	network_refresh	刷新网络统计。
6	service_refresh	刷新或查询 service manager。
7	display_guard	显示/UI 繁忙时降低预热强度。
8	thermal_guard	温度高时跳过 camera/media 预热。

模型负责选择策略，Android 端负责安全执行。比如 scheduler_bits=101010100 会请求 bit0、bit2、bit4、bit6 等动作，但 drop cache 仍然需要通过安全检查；如果当前内存不是 critical，ActionExecutor 会记录 skipped，而不是盲目执行。

5 实验设计

本文只讨论最终成功版本对应的实验。实验回答三个问题：

- 实时产品闭环是否能运行：连续采集、连续推理、刷新 Top-3 和执行调度。

- eBPF/OS 信息是否有助于预测：与只看 app 序列的 baseline 对比。
- Stage 3 调度是否真的影响系统指标：同一 Top-1 app 上比较无调度和有调度。

9 分钟实时 Top-3 实验在 rooted Pixel 5 上运行三个 3 分钟窗口。每个窗口真实启动设备上存在的 app，并收集自然产生的 eBPF/system evidence。每个窗口结束后由 MAPLE 推理并刷新 Top-3 和调度动作。该实验验证产品形态。

受控理论消融实验使用 Chengyu Fan 构造的 1000 条带 ground truth 的合成样本，逐组删除 app sequence、Binder、file、memory 等证据，观察多源 eBPF/OS 证据是否比 app sequence only 更有信息量。它先在可控数据上验证机制。

30 条真实记录预测实验使用 3 个 persona，每个 persona 10 条短 app 使用记录，共 30 条。Codex 根据 persona 生成“已观察 app 序列”和被隐藏的未来 Top-3 ground truth；执行阶段在 Android 设备上真实启动 app、发送 input 命令，并自然产生 raw eBPF。评估时模型看不到未来 Top-3，只看到输入 app 序列和压缩 OS/eBPF context。

系统调度性能实验不评价预测准确率，只评价 Stage 3 动作是否改善系统指标。实验先用真实 eBPF + MAPLE 选出同一个 Top-1 目标 app，本次为 QQ。Baseline 分支 force-stop QQ 后直接启动并测量；Stage 3 分支 force-stop QQ 后先执行同一组调度 bitmask，再启动同一个 QQ。这样比较对象一致，差别集中在是否执行 MAPLE 调度。

6 结果与分析

6.1 实时闭环结果

9 分钟实验的核心结果见表 2。三个窗口均产生约 1.9 万条 raw eBPF 事件和 52-55 个 app segment，说明实验不是手写 JSON，而是在手机上真实产生应用切换和系统事件。Top-3 在窗口间变化两次，说明推荐不是固定列表，而是随最近窗口更新。

表 2: 9 分钟实时 Top-3 实验结果

窗口	raw eBPF	app segments	MAPLE time	App id	Top-3
network	19263	52	29106 ms	220	Messages / Camera / Chrome
camera	19289	55	37196 ms	115	QQ / Messages / Camera
media_comm	19260	54	23839 ms	220	Messages / Camera / QQ

调度方面，前两个窗口输出 100000000，执行 Top-1 warm launch；第三个窗口输出 101010100，执行 widget update、warm launch、memory trim 和 service manager refresh，并因为内存不是 critical 安全跳过 drop cache。这说明 MAPLE 输出已经进入 Android 端动作执行层，而不是停留在文本建议。

6.2 预测消融结果

受控理论消融先验证 eBPF/OS 证据本身有用。1000 条合成样本中, Full eBPF 在 Qwen CPU 上达到 63.0%, App sequence only 为 30.1%, Random 为 4.8%。DeepSeek 配置下 Full eBPF 为 88.8%, App sequence only 为 51.1%。这说明多源 OS 证据在可控 ground truth 上提供了明显额外信息。

真实设备的 30 条记录实验得到同方向结果。表 3 展示最终成功配置: MEMO app+OS/eBPF 在 Top-1 Accuracy、Top-3 Hit、Precision@3、Recall@3、F1@3 和 MRR 上都高于 app-only baseline。提升幅度不夸张, 但方向与理论消融一致, 说明 eBPF/OS context 在真实设备路径中也能提供增益。

表 3: 30 条真实记录预测实验结果

配置	Top-1 Acc	Top-3 Hit	Precision@3	Recall@3	F1@3	MRR
App-only baseline	20.00%	46.67%	47.78%	47.78%	47.78%	31.11%
MEMO app+OS/eBPF	23.33%	50.00%	52.22%	52.22%	52.22%	34.44%

相对于 app-only baseline, Top-3 Hit 提升 3.33 个百分点, F1@3 提升 4.44 个百分点, Top-1 Accuracy 提升 3.33 个百分点, MRR 提升 3.33 个百分点。这里的结论应谨慎表述: eBPF/OS 并不是单独替代 app 序列, 而是作为系统上下文补充 app 行为预测, 并为后续调度提供证据。

6.3 系统调度性能结果

系统调度性能实验验证 Stage 3 是否真的改变系统指标。结果见表 4。在同一 MAPLE Top-1 app QQ 上, Stage 3 调度后 Launch TotalTime 从 433 ms 降到 168 ms, WaitTime 从 440 ms 降到 173 ms, 综合压力分数从 422.56 降到 9.71。CPU busy、iowait、reclaim 和 jank rate 也下降。

这组结果的关键不在于声称所有场景都会提升, 而在于证明 MAPLE 调度 bitmask 已经可以进入 ActionExecutor, 触发系统动作, 并在可测指标上留下结果。Stage 3 实际执行了 7 条动作记录, 其中包括 Top-1 warm launch、memory trim 和 service manager refresh; drop cache 被安全规则跳过, 因为当时内存不是 critical。

7 实现对应关系

表 4: 系统调度性能实验结果

指标	No scheduler	Stage 3	改善
Launch TotalTime	433 ms	168 ms	61.20%
WaitTime	440 ms	173 ms	60.68%
综合压力分数	422.56	9.71	97.70%
CPU busy	26.70%	20.79%	22.13%
iowait	0.094%	0.023%	75.42%
MemAvailable drop	164892 KB	-84448 KB	151.21%
reclaim	10046	0	100.00%
jank rate	1.240%	0.886%	28.56%

表 5: 关键实现文件

模块	作用
实时总控	RealtimePreloadController.kt: 管理持续采集、窗口提交、MAPLE worker 和调度执行。
窗口采集	RealtimeWindowRunner.kt: 负责每个窗口内的 app/eBPF/system evidence 收集和压缩。
历史记忆	RealtimeMemoryStore.kt: 维护有界 EMA memory, 避免历史上下文无限增长。
场景构造	MapleScenarioBuilder.kt: 构造 MAPLE scenario JSON, 并分离 app prediction candidates 与 scheduler evidence。
模型调用	MapleShellBackend.kt: 在 Android 端调用本地 MAPLE/llama.cpp 可执行文件并解析输出。
应用映射	AppIdMapping.kt: 扫描真实安装应用, 把 MAPLE id/category 映射成可启动 Top-3 app。
动作执行	ActionExecutor.kt: 执行调度 bitmask 对应的系统动作, 并记录执行/跳过原因。
桌面入口	MemoWidgetProvider.kt: 更新桌面 Widget, 展示可点击 Top-3 真实应用和最新更新时间。
性能实验	PressureExperimentRunner.kt: 执行系统调度性能实验, 比较同一目标 app 的无调度与 Stage 3 调度。

8 局限与下一步

当前系统仍有局限。第一，本地小模型不够“听话”，有时输出格式不完全符合 prompt。项目已经增加 parser 处理脏输出，但还不够 robust，后续应继续加强结构化输出约束和异常恢复。第二，实时性和轻量性存在 tradeoff。每 3 分钟推理一次可以让推荐和调度更及时，但后台持续采集和本地推理会消耗 CPU、电量和内存。第三，app scanner 对部分类别的映射还可以更精细，例如 camera 类场景应更稳定地选到真实相机应用。

下一步可以沿三个方向推进：其一，将 MAPLE 输出协议进一步结构化，减少自由文本解析；其二，增强 AppIdMapping 的自动分类能力，让不同手机上的真实应用映射更准确；其三，把调度动作从当前 warm launch、memory trim、service refresh 扩展到更细粒度的 Android 资源策略，同时继续保留安全门控，避免模型输出直接触发高风险系统操作。

9 结论

MEMO-Appflow 已经从 host 脚本原型推进为 Android 端侧产品闭环。它把 app 序列和 OS/eBPF 证据按时间对齐并压缩，交给本地 MAPLE 模型推理，再将结果转成真实 Top-3 app、桌面 Widget 和系统调度动作。实验表明，实时 Top-3 能随窗口更新；app+OS/eBPF 在真实记录预测中优于 app-only baseline；Stage 3 调度能在同一目标 app 上显著降低启动延迟和系统压力指标。更重要的是，这些结果来自手机内部运行的采集、推理和执行链路，而不是离线修改 JSON 或 host 侧脚本模拟。